

Distribuição de Conteúdo Multimédia

Serviços de Rede e Aplicações Multimédia

Mestrado em Engenharia de Telecomunicações e Informática

Ivo Miguel Menezes Silva

2º Semestre (25/26)

Multimedia Networking: Classes de Serviços

On demand

Conteúdo pedido pelo utilizador, como Netflix ou Spotify.

Permite atraso inicial e jitter mais elevados, porque o cliente pode usar buffer antes da reprodução.

Live

Conteúdo em direto, como transmissões de eventos ou aulas online.

Exige atraso e jitter relativamente pequenos para manter a sensação de atualidade.

Real-Time

Comunicação interativa, como chamadas de voz, videoconferência ou controlo remoto.

Requer atraso e jitter muito pequenos, porque qualquer demora afeta imediatamente a interação.

Em todos os casos, pode ser aceitável perder alguma informação, desde que a experiência de reprodução se mantenha fluida.

Multimedia Networking: Tipos de aplicações

Streaming audio, video armazenados

Streaming: a reprodução pode começar antes de transferir o ficheiro completo.

Conteúdo armazenado no servidor: pode ser transmitido mais depressa do que o áudio ou vídeo é reproduzido, o que implica armazenamento temporário e buffering no cliente.

Exemplos: YouTube, Netflix, Disney+.

Comunicação de voz e vídeo interativa sobre IP

A natureza interativa da conversa entre pessoas limita a tolerância ao atraso. O objetivo principal não é só ouvir ou ver, mas manter a interação em tempo real, daí a baixa tolerância a atrasos.

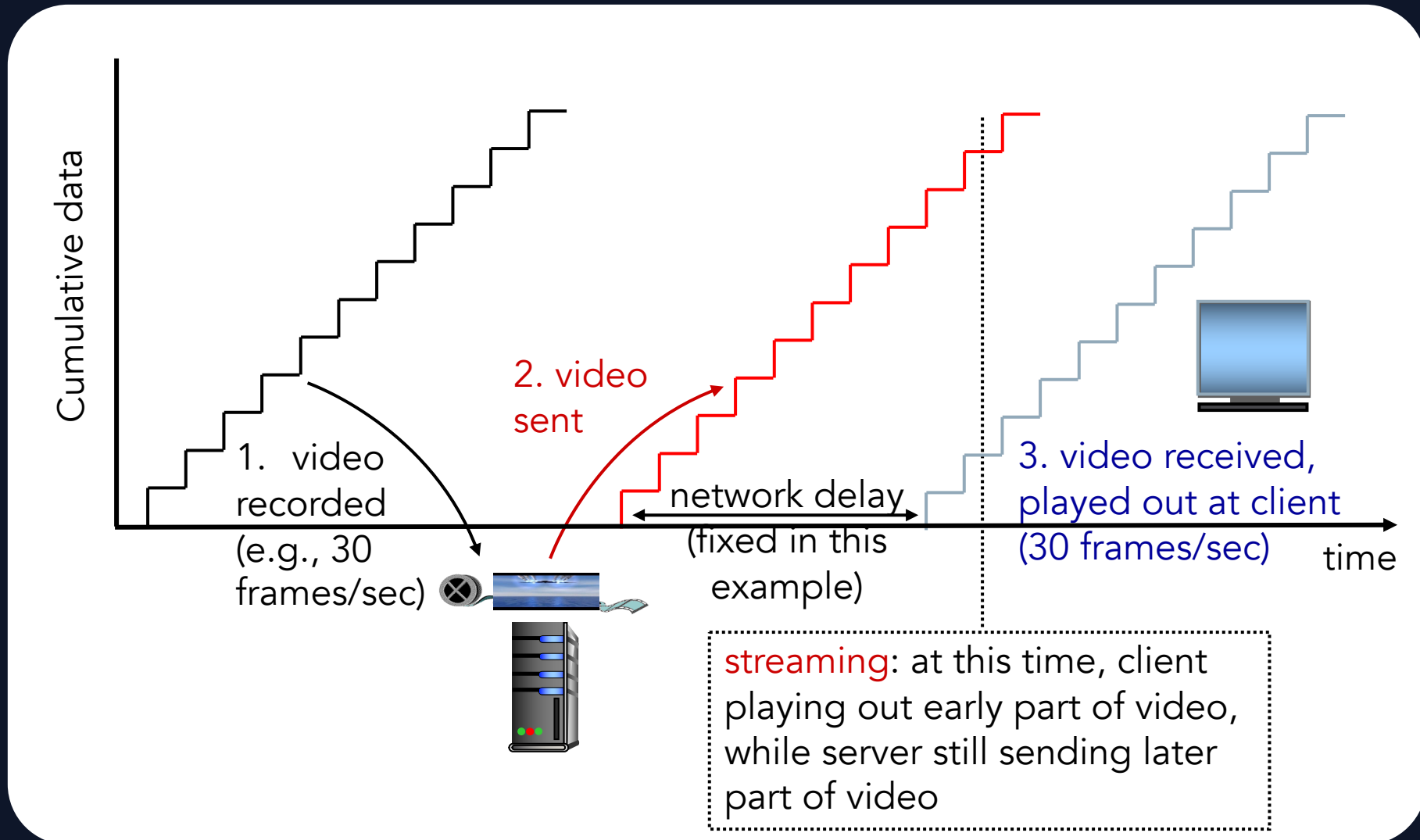
Exemplos: Teams, Zoom, WhatsApp.

Transmissão em direto (streaming live) de áudio, vídeo

O sistema precisa de equilibrar atraso baixo com alguma margem de buffering para lidar com *jitter* e perdas.

Exemplos: Transmissão de um evento desportivo ao vivo

Multimedia Networking: Streaming de vídeo armazenado



Streaming de vídeo armazenado: Desafios



Restrição de reprodução contínua

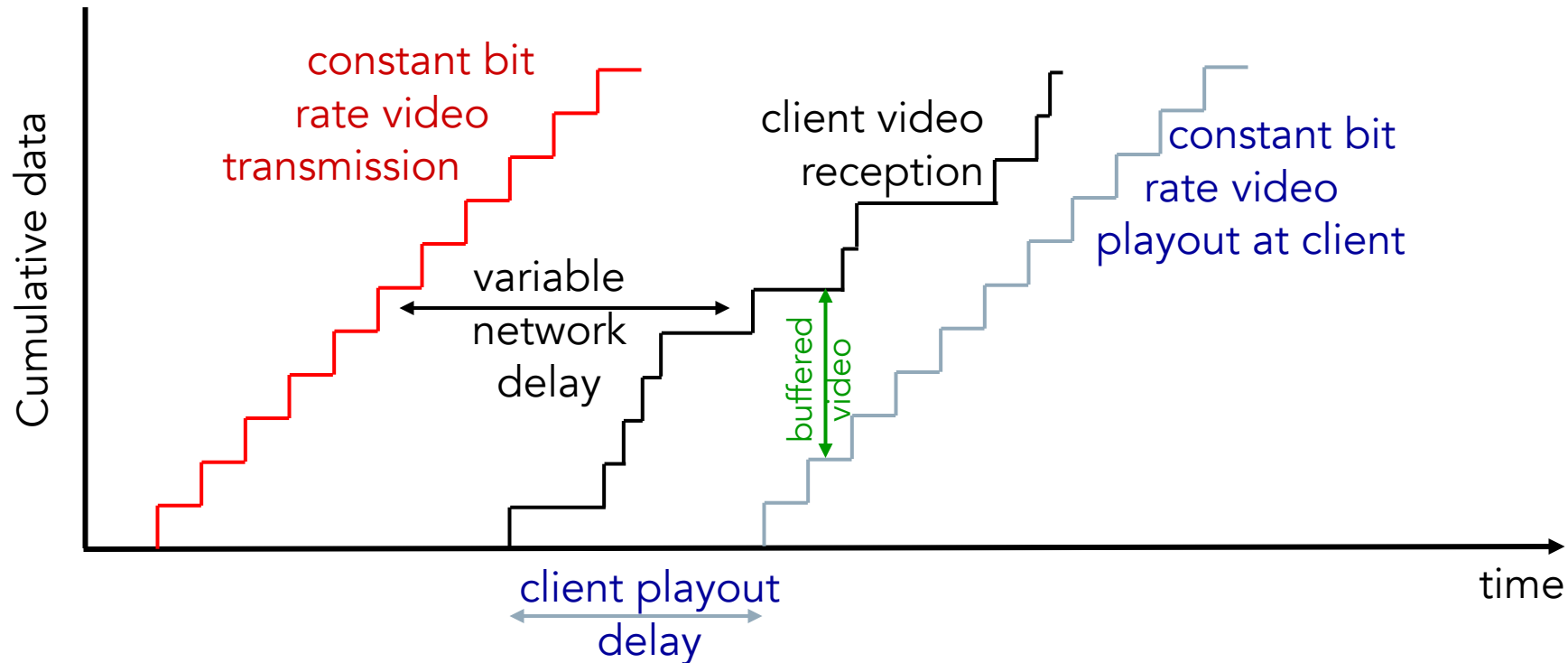
- Quando a reprodução no cliente começa, a apresentação tem de respeitar o tempo original do vídeo.
 - No entanto, os **atrasos de rede são variáveis**, ou seja, existe jitter.
 - Por isso, é necessário um **buffer no cliente** para cumprir os requisitos de reprodução.



Outros desafios

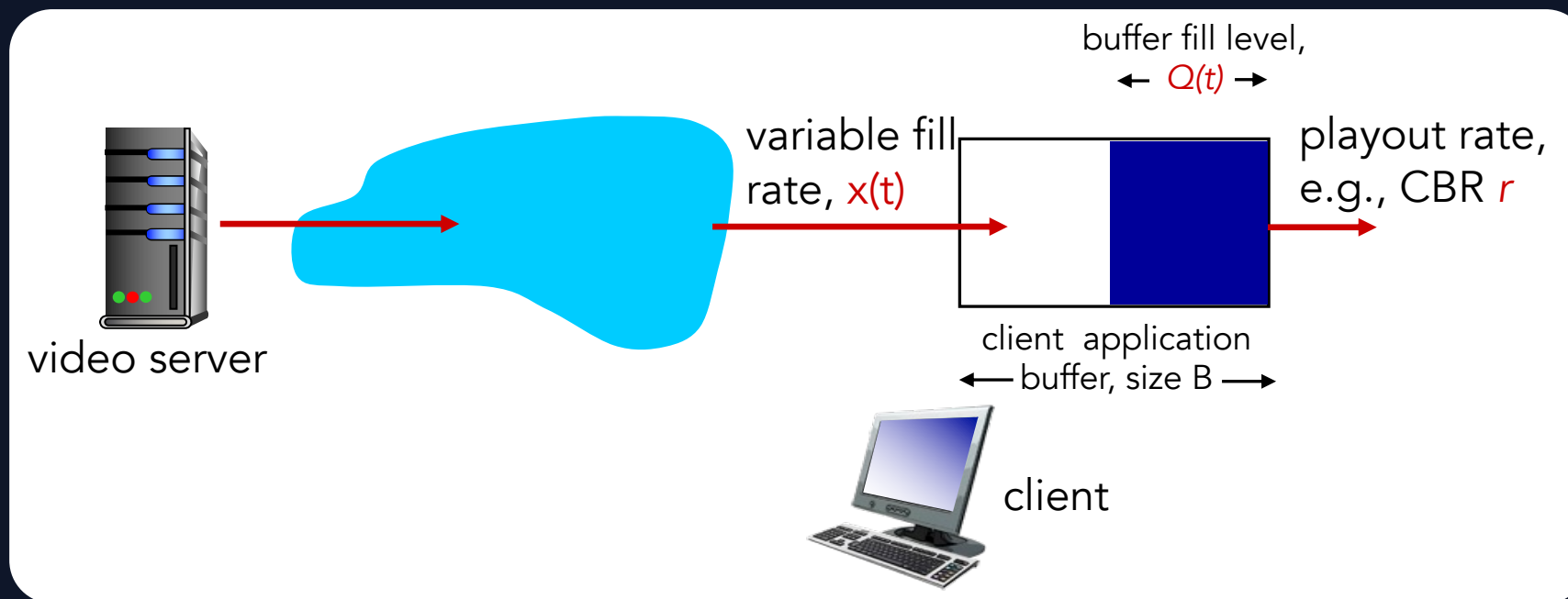
- **Interatividade do cliente:** pausa, avanço rápido, retrocesso e salto para outra parte do vídeo.
- Os pacotes de vídeo podem perder-se e ser retransmitidos.

Multimedia Networking: Streaming de vídeo armazenado



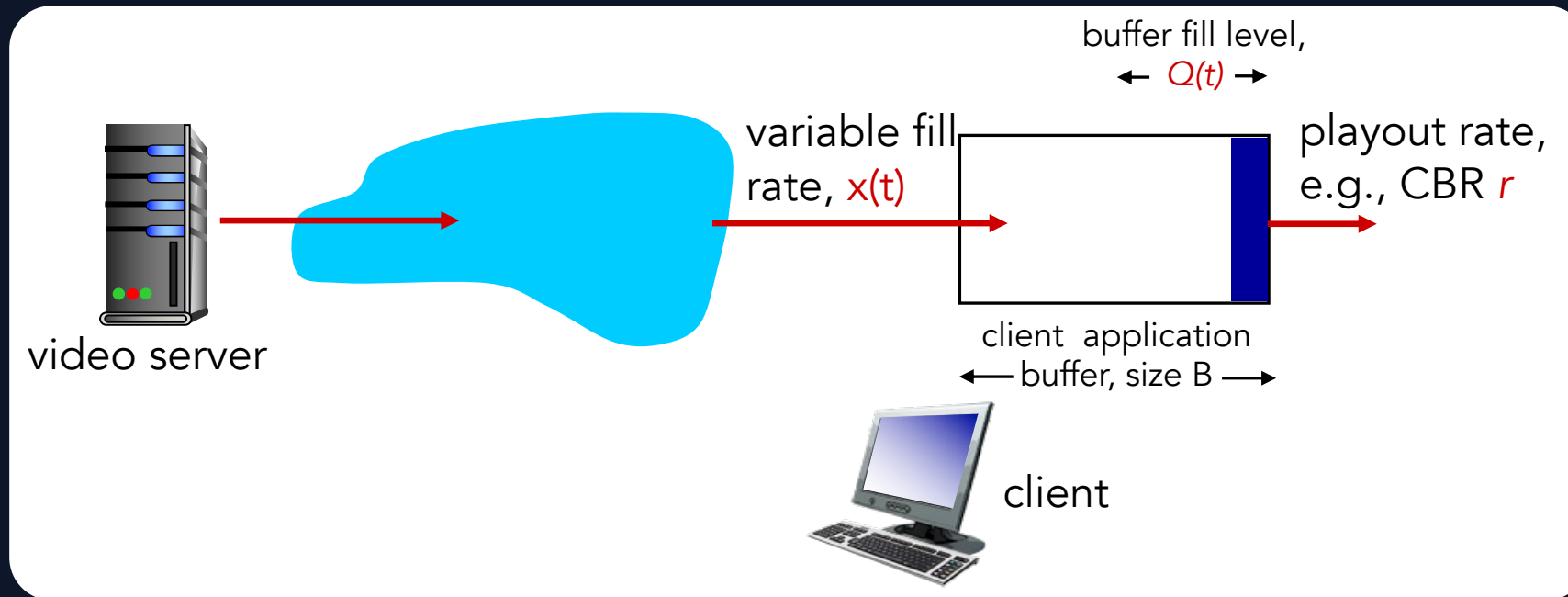
client-side buffering and playout delay:
compensate for network-added delay, delay jitter

Client-side buffering, playout



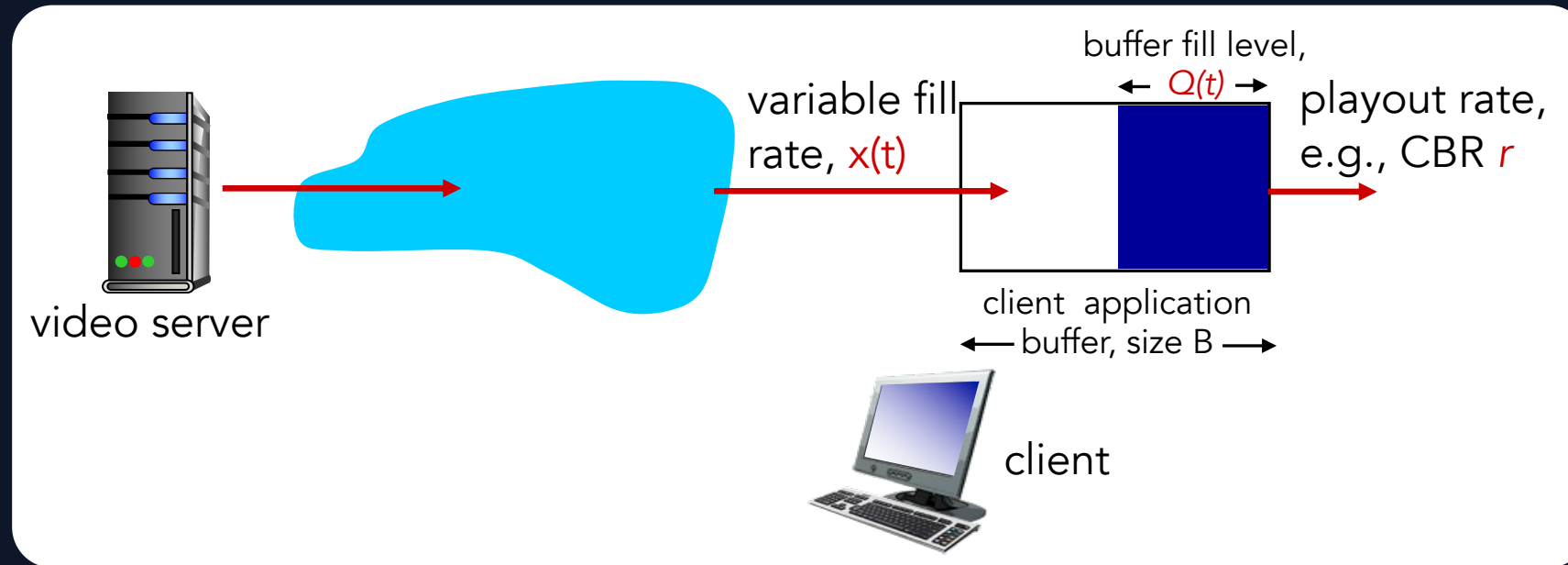
- 🎥 À esquerda temos o servidor de vídeo.
- ☁ Os dados atravessam a rede, representada pela nuvem azul, onde a taxa de chegada varia no tempo, definida por $x(t)$. No cliente, esses dados entram num buffer de aplicação, de tamanho B , cujo nível é $Q(t)$.
- ▶ A reprodução sai desse buffer a uma taxa constante, r , o que permite compensar a variabilidade da rede.

Client-side buffering, playout



1. Enchimento inicial do buffer até ao início da reprodução no instante t_p
2. Reprodução inicia no instante t_p
3. O nível de enchimento do buffer varia ao longo do tempo, uma vez que a taxa de enchimento $x(t)$ varia, e a taxa de reprodução, r , é constante

Client-side buffering, playout



Playout: taxa de enchimento média ($\bar{x}$), taxa de reprodução (r):

- Se $\bar{x} < r$: o buffer acaba por esvaziar, causando interrupção da reprodução até voltar a encher.
- Se $\bar{x} > r$: o buffer não esvazia, desde que o atraso inicial de reprodução seja suficientemente grande para absorver a variabilidade em $x(t)$

Atraso inicial de reprodução tem que ser suficiente para absorver as variações instantâneas em $x(t)$. Essa margem inicial funciona como uma reserva de segurança para lidar com jitter e com flutuações temporárias da rede.

Streaming de Multimedia: UDP

- O servidor envia os dados à taxa adequada para o cliente.
 - Muitas vezes temos:
 - **taxa de envio = taxa de codificação = taxa de consumo**
 - A taxa de transmissão pode ser independente do estado de congestionamento da rede.
- Usa-se um *playout delay* curto (1 a 2 segundos), para remover o *jitter* de rede.
- A **recuperação de erros** é feita, se necessário, ao nível da aplicação.
- Requer:
 - RTP/RTCP [RFC 3550]: para transportar tipos de multimédia.
 - RTSP [RFC 2326]: Real Time Streaming Protocol para controlar a sessão, por exemplo pausar, retomar ou reposicionar o vídeo.

Streaming de Multimedia: UDP

Principais desvantagens:

- Se a largura de banda descer abaixo da taxa de codificação, pode haver congelamento da imagem, **perda de frames** e **má experiência para o utilizador** (a aplicação tem de se adaptar a essa situação).
- Controlo adicional: através de uma ligação RTSP separada para gerir ações do cliente, como pausa ou retoma.
- O UDP pode não atravessar *firewalls*, porque *middleboxes* podem bloquear esse tráfego e impedir a receção do vídeo.

Isto torna o desenvolvimento em larga escala mais difícil.

Streaming de Multimedia: HTTP

- Evita a necessidade de um servidor de controlo de media.
- A terminação e o reposicionamento do vídeo fazem-se com um novo pedido HTTP GET.
- O cabeçalho HTTP byte-range permite indicar o novo bloco de bytes a partir do qual a reprodução deve continuar.
- Se o utilizador reposicionar o vídeo em t_0 para um instante futuro t , com ($t > t_0 + B/r$), o conteúdo do buffer da aplicação, B bits (e correspondente largura de banda), é desperdiçado.
- HTTP/TCP atravessa mais facilmente *firewalls*.

Estas vantagens **reduzem o custo de desenvolvimento** em larga escala na Internet.

Streaming de Multimedia: DASH

def O que é o protocolo DASH?

O protocolo MPEG-DASH (Dynamic Adaptive Streaming over HTTP) é uma tecnologia de streaming adaptável que opera na camada de aplicação (usando HTTP/TCP ou HTTP/3/UDP), cuja principal vantagem é entregar vídeo de alta qualidade com o mínimo de interrupções (buffering), ajustando automaticamente a resolução em tempo real conforme a velocidade da internet do utilizador.

Streaming de Multimedia: DASH

Servidor

- O servidor divide o ficheiro de vídeo em vários *chunks**.
- Cada *chunk* existe em várias taxas de codificação.
- As diferentes versões são guardadas em ficheiros distintos.
- Esses ficheiros são replicados em vários nós da CDN.
- Um manifest file fornece os URLs dos *chunks*.

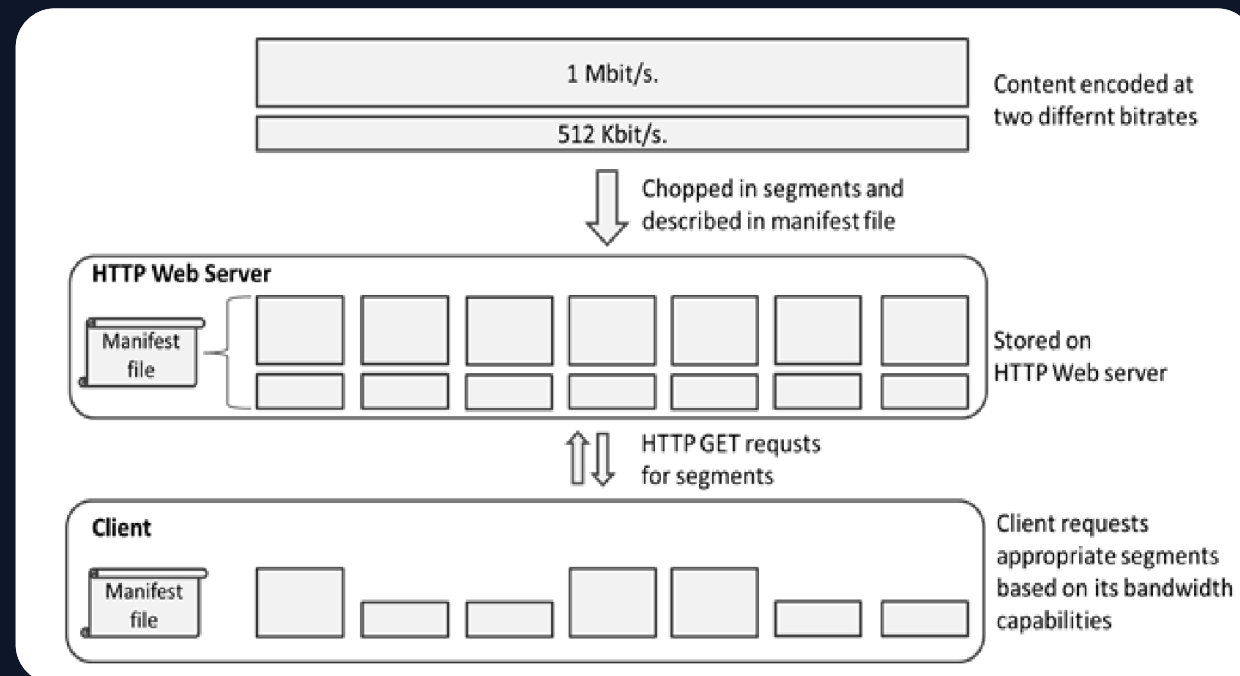
Cliente

- Mede periodicamente a largura de banda do servidor para o cliente.
- Consultando o manifest, pede um chunk de cada vez.
- Escolhe a maior taxa de codificação que a largura de banda atual suporta.
- Pode mudar de taxa ao longo do tempo e também mudar de servidor, conforme a largura de banda disponível.

**chunks*: são uma pequena parte (bloco/segmento) de um ficheiro áudio/vídeo (geralmente com poucos segundos de duração).

Streaming de Multimedia: DASH

1. O DASH divide o vídeo em pequenos segmentos. Cada segmento existe em várias qualidades, por exemplo com diferentes taxas de codificação.
2. O cliente consulta um ficheiro de manifesto, que indica onde estão os segmentos e quais as versões disponíveis. Depois, mede a largura de banda disponível e pede, um a um, os segmentos que consegue sustentar.
3. Se a rede estiver rápida, escolhe uma qualidade mais alta. Se a largura de banda baixar, passa para uma qualidade mais baixa. Assim, o vídeo adapta-se dinamicamente às condições da rede e a reprodução torna-se mais estável.



Streaming de Multimedia: DASH

“Inteligência” está no cliente; o cliente determina:

- **quando** pedir um *chunk*, para evitar esvaziamento ou *overflow* do buffer.
- **que taxa de codificação** pedir, escolhendo maior qualidade quando houver mais largura de banda.
- **onde** pedir o *chunk*, podendo escolher um servidor próximo ou com melhor largura de banda disponível.
- Também conhecido como **MPEG-DASH**, definido na norma ISO/IEC 23009-1:2019.
- É equivalente ao **HLS (HTTP Live Streaming)**, introduzido pela Apple em 2009 e normalizado em RFC 8216 em 2017.

Voice-over-IP (VoIP)



Requisito de atraso de ponta a ponta no VoIP: necessário para manter o aspeto «conversacional»

- Atrasos mais elevados são perceptíveis e prejudicam a interatividade
- **< 150 ms: bom** 👍
- **> 300 ms: mau** 👎
- inclui atrasos ao nível da aplicação (*pacotização, reprodução*) e da rede

↔ **inicialização da sessão:** como é que o destinatário da chamada divulga o endereço IP, o número da porta e os algoritmos de codificação?

■ **serviços de valor acrescentado:** reencaminhamento de chamadas, redirecionamento, gravação, etc.

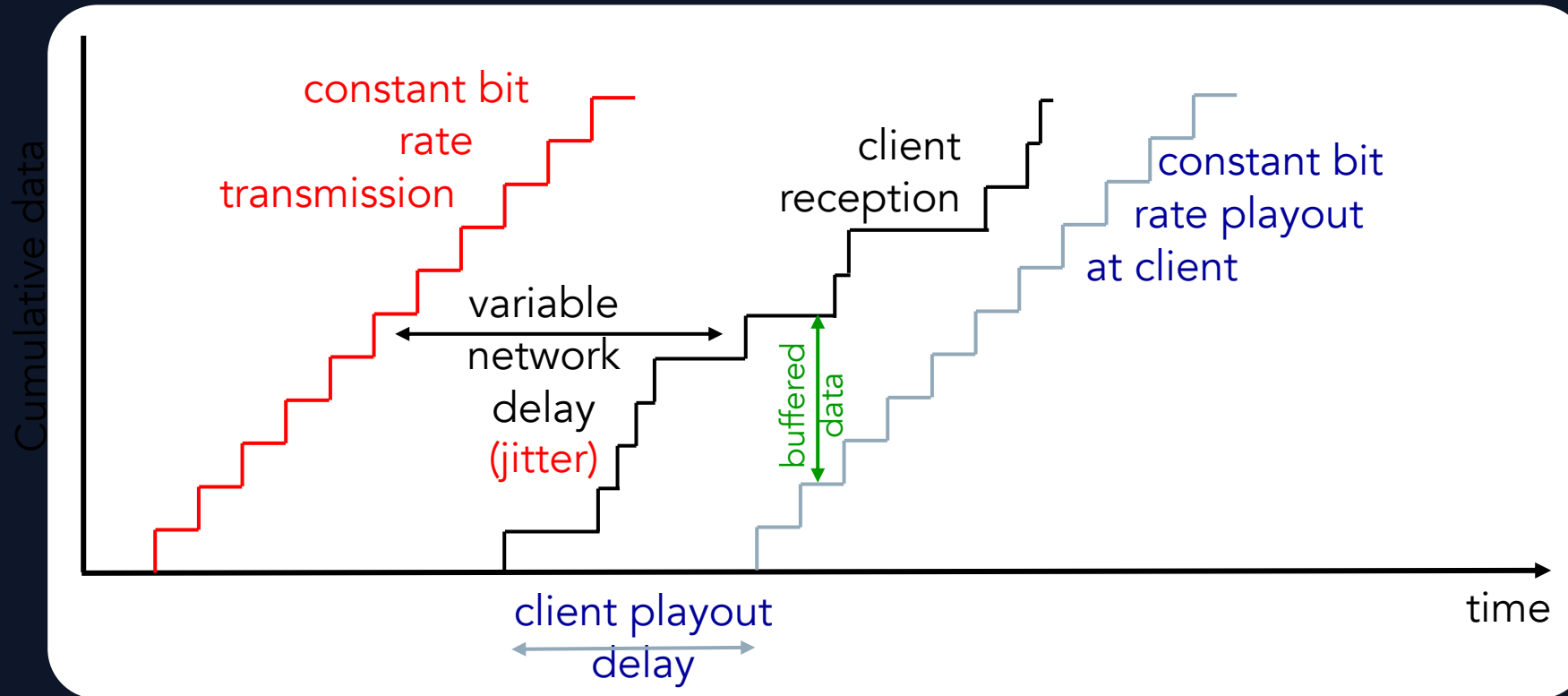
Características do VoIP

- **Áudio do orador:** alternância entre intervalos de fala e períodos de silêncio.
 - 64 kbps durante os intervalos de fala
 - Pacotes gerados apenas durante os intervalos de fala
 - Blocos (*chunks*) de 20 ms a 8 kbytes/s: 160 bytes de dados
- Cabeçalho da camada de aplicação adicionado a cada *chunk*
- Chunk + cabeçalho são encapsulados em UDP (ou segmento TCP)
- A aplicação envia um segmento para o *socket* a cada 20 ms durante os intervalos de fala

VoIP: perda de pacotes, atraso

- **Recordemos:** por predefinição, o IP fornece um serviço de «*best-effort*»
- **Perda na rede:** datagrama IP perdido devido a congestionamento da rede (transbordamento do buffer do router)
- **Perda por atraso:** o datagrama IP chega demasiado tarde para ser reproduzido no recetor
 - **atrasos:** processamento, filas (*queueing*) na rede; atrasos no sistema final (remetente, recetor)
 - atraso máximo tolerável típico: **300 ms**
 - um pacote atrasado é um pacote perdido
- **Tolerância à perda:** dependendo da codificação de voz, podem ser toleradas taxas de perda de pacotes entre 1% e 10%

Delay Jitter



© iSilva 2026

Efeito de delay jitter em VoIP

Embora os pacotes sejam transmitidos a intervalos regulares, chegam ao recetor com atrasos variáveis, o que obriga ao uso de um buffer e de um atraso inicial de reprodução para garantir reprodução a taxa constante.

VoIP: Atraso de reprodução fixo

Quando iniciar a reprodução?

- O recetor tenta reproduzir cada bloco exatamente **q** milissegundos após a sua geração.
 - Se o bloco tiver o timestamp **t** : reproduzir o bloco em **t + q**
 - Se o bloco chegar após **t + q**: os dados chegam demasiado tarde para a reprodução: os dados são «perdidos»
- Compromisso na escolha de **q**:
 - **q elevado**: menor perda de pacotes
 - **q baixo**: melhor experiência interativa

VoIP: Atraso de reprodução fixo

O emissor gera pacotes a cada 20 ms durante um *talk spurt*.

1º pacote

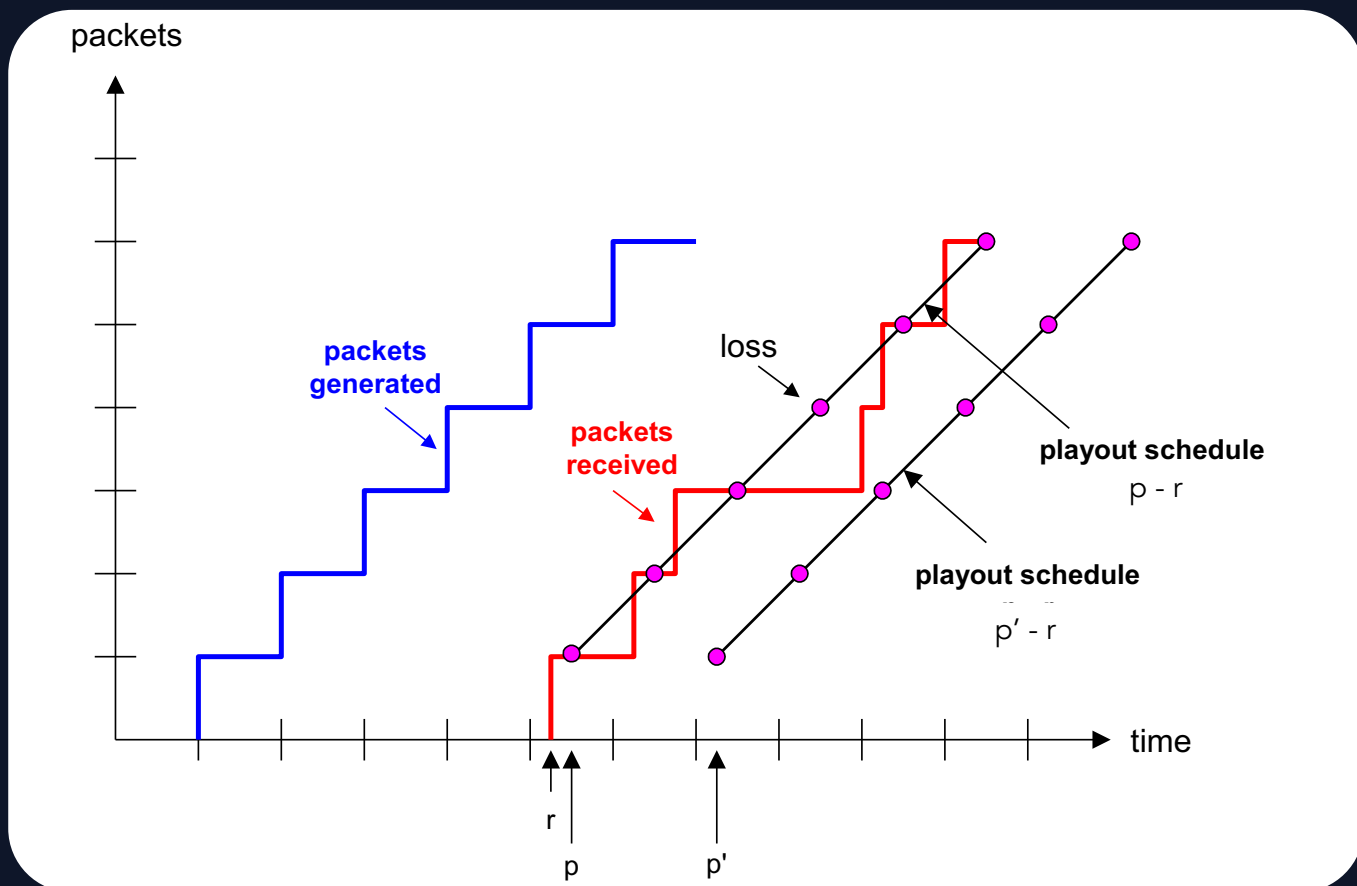
- recebido no instante r .

1ª programação de reprodução

- começa no instante p .

2ª programação de reprodução

- começa no instante p'



VoIP: Atraso de reprodução adaptativo (I)

Objetivo: baixo atraso de reprodução e baixa taxa de perdas por chegada tardia.

Abordagem: atraso de reprodução adaptativo

- estimar o atraso da rede e ajustar a reprodução no início de cada *talk spurt*;
- os períodos de silêncio podem ser comprimidos ou alongados;
- durante o *talk spurt*, os *chunks* continuam a ser reproduzidos a cada 20 ms.

Estimar adaptativamente o **atraso dos pacotes** com EWMA (*Exponentially Weighted Moving Average*):

$$d_i = (1 - \alpha)d_{i-1} + \alpha(r_i - t_i)$$

 Delay estimate after <i>i</i> -th packet	 small constant, e.g. 0.1	 time received	 time sent (timestamp)
		{ measured delay of <i>i</i> th packet	

VoIP: Atraso de reprodução adaptativo (II)

- Também é útil estimar o **desvio médio do atraso**, v_i :

$$v_i = (1-\beta)v_{i-1} + \beta |r_i - t_i - d_i|$$

- As estimativas d_i e v_i são calculadas para cada pacote recebido, mas usadas apenas no início de cada talk spurt.
- Para o primeiro pacote de um talk spurt, o instante de reprodução é:

$$\text{playout-time}_i = t_i + d_i + Kv_i$$

- Os restantes pacotes do talk spurt são reproduzidos periodicamente.
- Ou seja,
 - d_i representa o atraso médio da rede.
 - v_i estima o desvio médio desse atraso, ou seja, a sua variabilidade.
 - Kv_i serve para transformar essa variabilidade numa folga temporal antes da reprodução.

Quanto maior for K ou v_i , maior será o atraso de playout escolhido, reduzindo perdas tardias mas aumentando latência.

VoIP: Atraso de reprodução adaptativo (III)

Como é que o recetor sabe que começou um novo intervalo de transmissão?

Como é que determina se um pacote é o primeiro num talk spurt?

- **Sem perdas:** o recetor analisa os **timestamps sucessivos**
 - **Se** a diferença for superior a 20 ms, começa um novo intervalo de transmissão.
- **Com possíveis perdas:** o recetor verifica os **timestamps** e os **números de sequência**
 - **Se** diferença timestamps sucessivos for superior a 20 ms **e se** os números de sequência sem falhas, indica um novo intervalo de transmissão.

VoIP: Recuperação da perda de pacotes (I)

Desafio: recuperar perdas de pacotes sem introduzir demasiado atraso na reprodução.

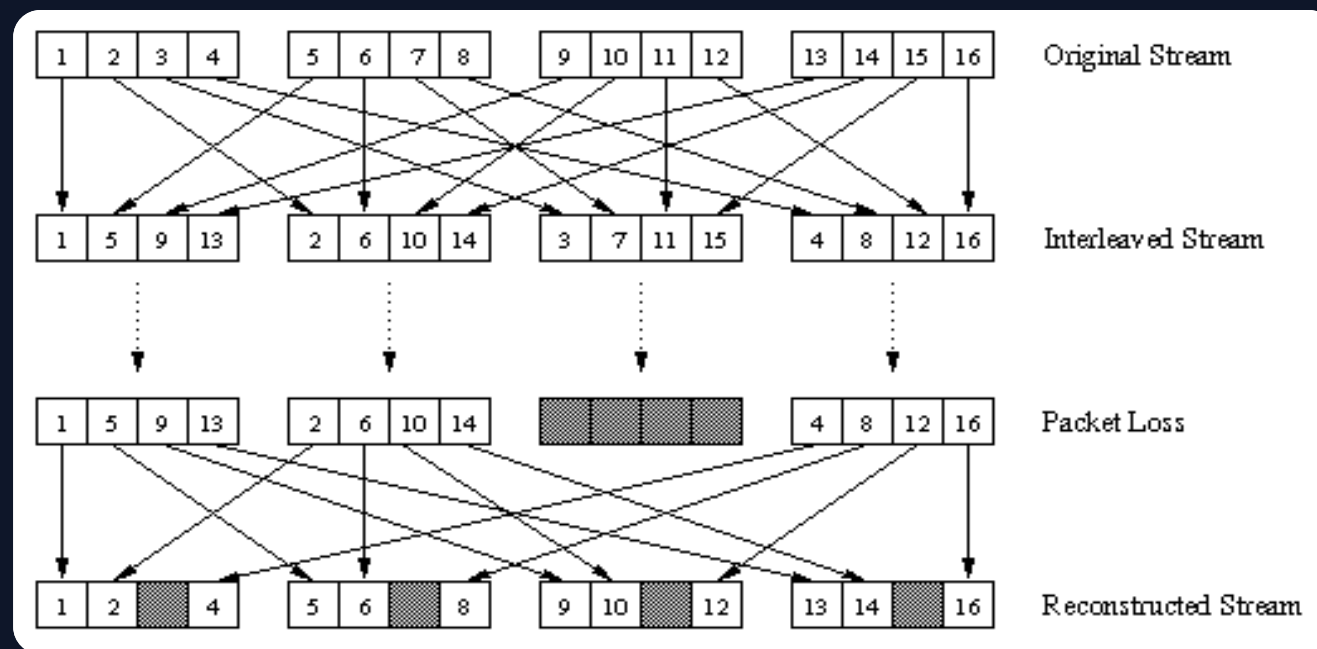
- Como cada ACK/NAK custa cerca de um RTT, a retransmissão pode ser demasiado lenta para aplicações em tempo real.
- Uma alternativa é usar **FEC (Forward Error Correction)**
 - Enviando redundância extra para que o recetor consiga recuperar perdas sem pedir retransmissão.
- **Simple FEC**
 - Por cada n blocos de áudio, envia-se um bloco redundante obtido por XOR, o que acrescenta uma sobrecarga de $1/n$ na largura de banda (porque se enviam $n+1$ blocos).
 - Se apenas um dos $n+1$ blocos se perder, o recetor consegue reconstruí-lo, com um atraso na reprodução

VoIP: Recuperação da perda de pacotes (II)

Interleaving para ocultar perdas:

- Os blocos de áudio são divididos em unidades mais pequenas, por exemplo, quatro unidades de 5 ms por cada bloco de áudio de 20 ms.
- Cada pacote contém pequenas unidades provenientes de blocos diferentes.
- Se um pacote se perder, continua a existir a maior parte de cada bloco original.

Não há sobrecarga de redundância, mas **aumenta a latência** e o **atraso de reprodução**.



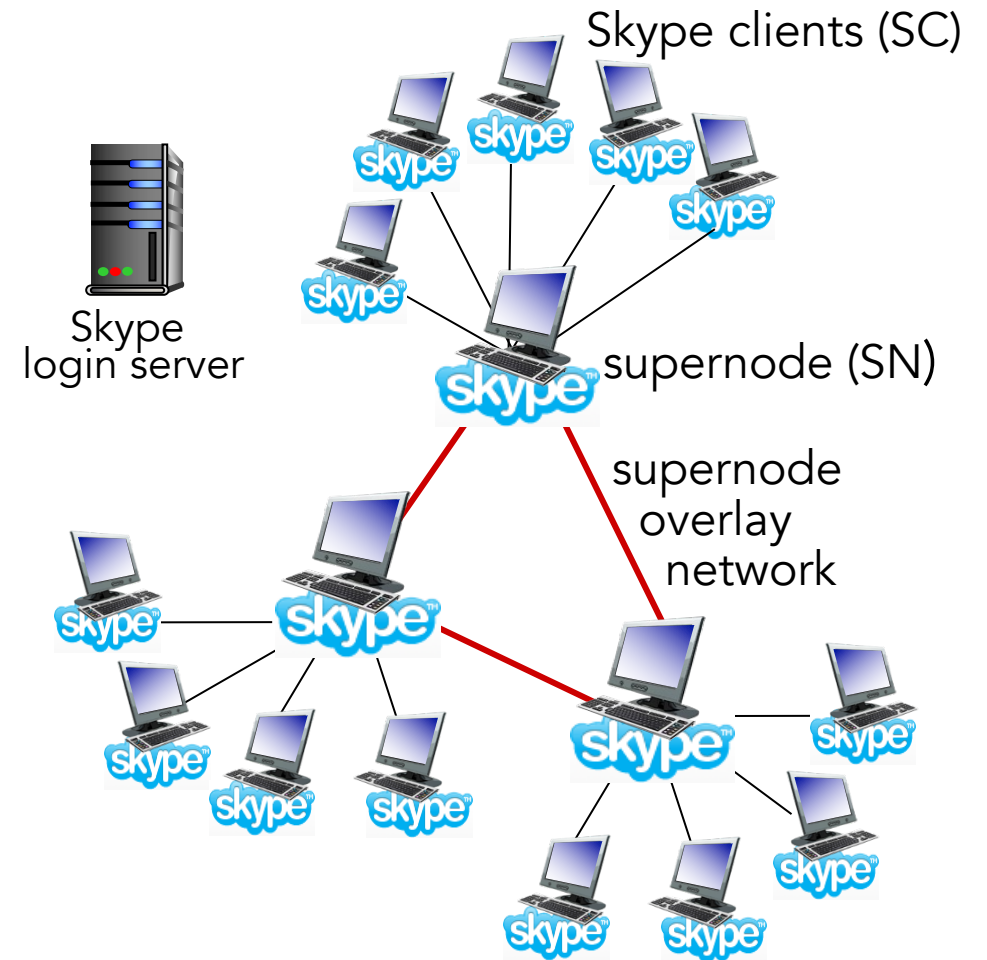
Voice-over-IP: Skype

Protocolo proprietário da camada de aplicação (deduzido através de engenharia reversa)

- mensagens encriptadas

Componentes P2P:

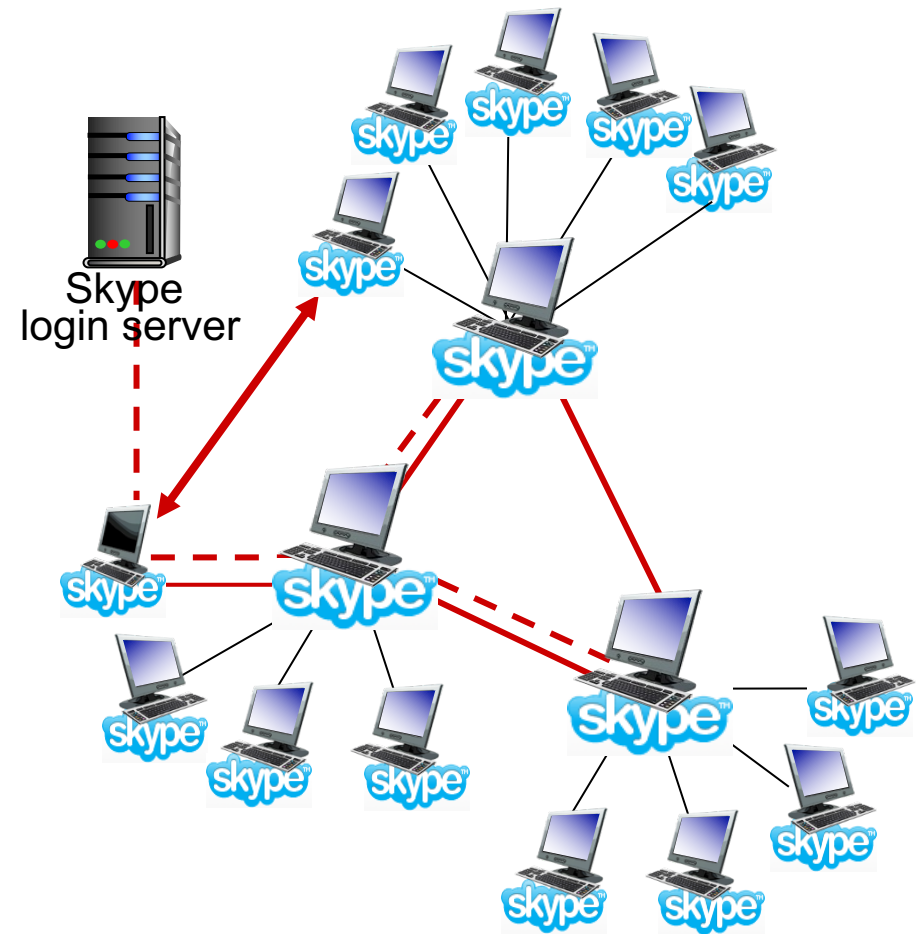
- **clientes:** os utilizadores do Skype ligam-se diretamente uns aos outros para efetuar chamadas VoIP
- **supernodes (SN):** utilizadores do Skype com funções especiais
- **Overlay network:** entre os SN para localizar os SC
- **servidor de login**



Voice-over-IP: Skype

Funcionamento do cliente Skype:

1. Entra na rede Skype contactando um SN cujo endereço IP está em cache, usando TCP.
2. Faz autenticação (*login*), com nome de utilizador e palavra-passe, num servidor centralizado de login.
3. Obtém o endereço IP do destinatário a partir do SN, da overlay de SNs, ou da lista de contactos do cliente.
4. Inicia a chamada diretamente para o destinatário.



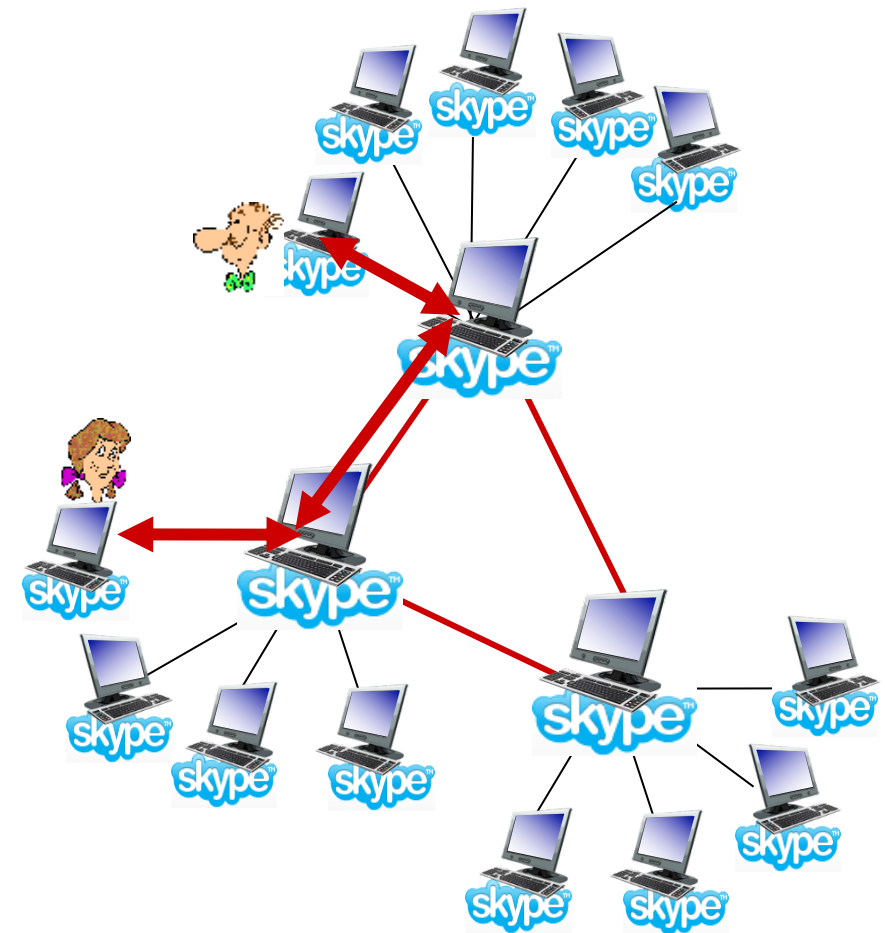
Skype: Pares como retransmissores

Problema: tanto a Alice como o Bob estão atrás de "NATs".

- O NAT impede que um *host* externo inicie uma ligação para um *host* interno.
- Um *host* interno pode iniciar uma ligação para o exterior.

Solução com relay: a Alice e o Bob mantêm uma ligação aberta aos seus SNs.

- A Alice sinaliza ao seu SN para estabelecer ligação com o Bob.
- O SN da Alice liga-se ao SN do Bob.
- O SN do Bob liga-se ao Bob usando a ligação aberta que ele próprio tinha iniciado anteriormente para o seu SN.



Bibliografia

- Kurose, J. F., & Ross, K. W. (2017). *Computer networking: A top-down approach* (7th ed.). Addison-Wesley.